

DiceKriging vs RobustGaSP

1D test example

We have a 1-dimensional input, one output, and 5 noise-free observations, obtained using the function $f(x) = 2 \sin(3x - 0.5)$:

$$x = \{0.1, 0.3, 0.5, 0.7, 0.9\}$$
$$y = \{-0.4, 0.78, 1.68, 2, 1.62\}$$

which is plotted in Figure 1.

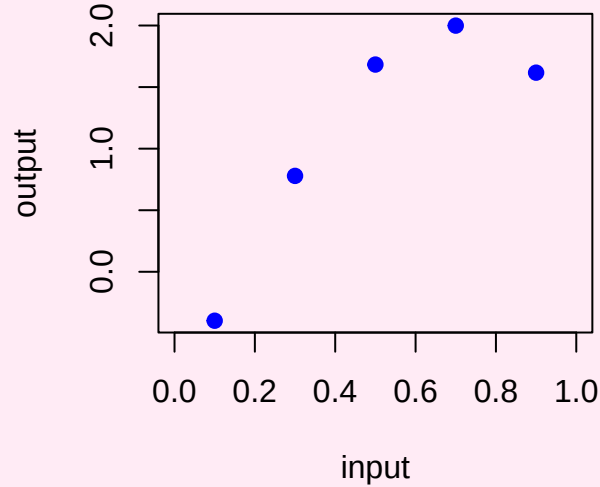


Figure 1: 1D example observations

Going forward, we will assume a linear mean function of the form

$$m(x) = \beta_0 + \beta_1 x.$$

In RobustGaSP's `rgasp()` function, this involves specifying the **trend** matrix H :

```
##      [,1] [,2]
## [1,]    1  0.1
## [2,]    1  0.3
## [3,]    1  0.5
## [4,]    1  0.7
## [5,]    1  0.9
```

as well as setting `zero.mean` equal to "No". The Gauss (squared exponential) *and* Matern 5/2 kernels are compared for both packages, leading to the creating of four functions, `gp_km_gauss()`, `gp_km_matern52()`, `gp_rgasp_gauss()` and `gp_rgasp_matern()`, reprinted below:

```

gp_km_gauss <- function(input_matrix, output_matrix){
  km(~.,
    input_matrix,
    output_matrix,
    covtype="gauss",
    optim.method="BFGS",
    control=list(maxit=500)
  )
}

gp_km_matern52 <- function(input_matrix, output_matrix){
  km(~.,
    input_matrix,
    output_matrix,
    covtype="matern5_2",
    optim.method="BFGS",
    control=list(maxit=500)
  )
}

gp_rgasp_gauss <- function(input_matrix, output_matrix, H_matrix){
  rgasp(design = input_matrix,
    response = output_matrix,
    trend = H_matrix,
    lower_bound=FALSE,
    kernel_type = 'pow_exp', alpha = rep(2, nrow(input_matrix)),
    zero.mean = "No"
  )
}

gp_rgasp_matern52 <- function(input_matrix, output_matrix, H_matrix){
  rgasp(design = input_matrix,
    response = output_matrix,
    trend = H_matrix,
    lower_bound=FALSE,
    zero.mean = "No"
  )
}

```

Note that `lower_bound` is set to `FALSE` - Jonathan doesn't *think* this is incorrect.

The output from the two RobustGaSP functions is shown below (first for the Gauss kernel and second for the Matern 5/2).

```

## The upper bounds of the range parameters are Inf
## The initial values of range parameters are 0.001124683
## Start of the optimization 1 :
## The number of iterations is 44
## The value of the marginal posterior function is -0.2280412
## Optimized range parameters are 3.655377
## Optimized nugget parameter is 0
## Convergence: TRUE
## The initial values of range parameters are 0.064
## Start of the optimization 2 :

```

```

## The number of iterations is 25
## The value of the marginal posterior function is 4.466659
## Optimized range parameters are 1.114853
## Optimized nugget parameter is 0
## Convergence: TRUE

## The upper bounds of the range parameters are Inf
## The initial values of range parameters are 0.001124683
## Start of the optimization 1 :
## The number of iterations is 21
## The value of the marginal posterior function is -5.227341
## Optimized range parameters are 619828629467
## Optimized nugget parameter is 0
## Convergence: TRUE
## The initial values of range parameters are 0.064
## Start of the optimization 2 :
## The number of iterations is 16
## The value of the marginal posterior function is 3.12738
## Optimized range parameters are 4.677144
## Optimized nugget parameter is 0
## Convergence: TRUE

```

We believe there are two optimisations since the first is getting a starting point for the second. Jonathan doesn't think it's a problem that we sometimes get a **FALSE** result for **Convergence**.

To visualise the posterior mean functions returned by these four functions, predictions are made at 1000 new points on the range $[0, 1]$. Once again, **rgasp** requires specification of the 'trend' matrix H , this time containing the testing inputs.

These 1000 points are plotted in Figure 2 (red line), along with 95% credible intervals (red dashed line) along with the actual function (black line) and 5 observations (blue points).

Various checks are performed (as in the example from **RobustGaSP**'s Help page):

Table 1: RobustGaSP manual's suite of checks

Package	Kernel	Prop. w/n 95% CI	Out-of-sample MSE	Mean width of 95% CI
DiceKriging	Gauss	1	0.001	0.0739
DiceKriging	Matern 5/2	0.82	0.5	0.7133
RobustGaSP	Gauss	1	0.0004	0.1577
RobustGaSP	Matern 5/2	1	0.03	0.6329

Finally, in preparation for the real world data where we cannot perform out-of-sample validation, we'll perform leave-one-out validation: for each of the observations, fitting a GP using all observations except this one, then using the fitted emulator to predict the output at the excluded observation's input setting. The 95% credible interval for each prediction is checked against the actual output value, and the proportion of these CI's that contain the respective observation is found. The results are shown in Tables 2-5 below.

Table 2: DiceKriging with the Gauss kernel (60% validation)

	actual	loo_prediction	lower95	upper95	validated
1	-0.40	0.81	0.18	1.44	0
3	1.68	1.00	0.22	1.77	1

	actual	loo_prediction	lower95	upper95	validated
4	2.00	1.65	1.17	2.14	1
5	1.62	3.04	2.61	3.47	0

Table 3: DiceKriging with the Matern 5/2 kernel (60% validation)

	actual	loo_prediction	lower95	upper95	validated
1	-0.40	0.81	0.18	1.44	0
3	1.68	1.00	0.22	1.77	1
4	2.00	1.59	0.81	2.37	1
5	1.62	3.04	2.61	3.47	0

Table 4: RobustGaSP with the Gauss kernel (80% validation)

	actual	loo_prediction	lower95	upper95	validated
1	-0.40	-0.58	-0.65	-0.51	0
3	1.68	1.67	1.62	1.72	1
4	2.00	2.00	1.87	2.12	1
5	1.62	1.78	1.14	2.43	1

Table 5: RobustGaSP with the Matern 5/2 kernel (100% validation)

	actual	loo_prediction	lower95	upper95	validated
1	-0.40	-0.64	-1.20	-0.09	1
3	1.68	1.67	1.39	1.95	1
4	2.00	1.96	1.52	2.41	1
5	1.62	1.94	0.59	3.28	1

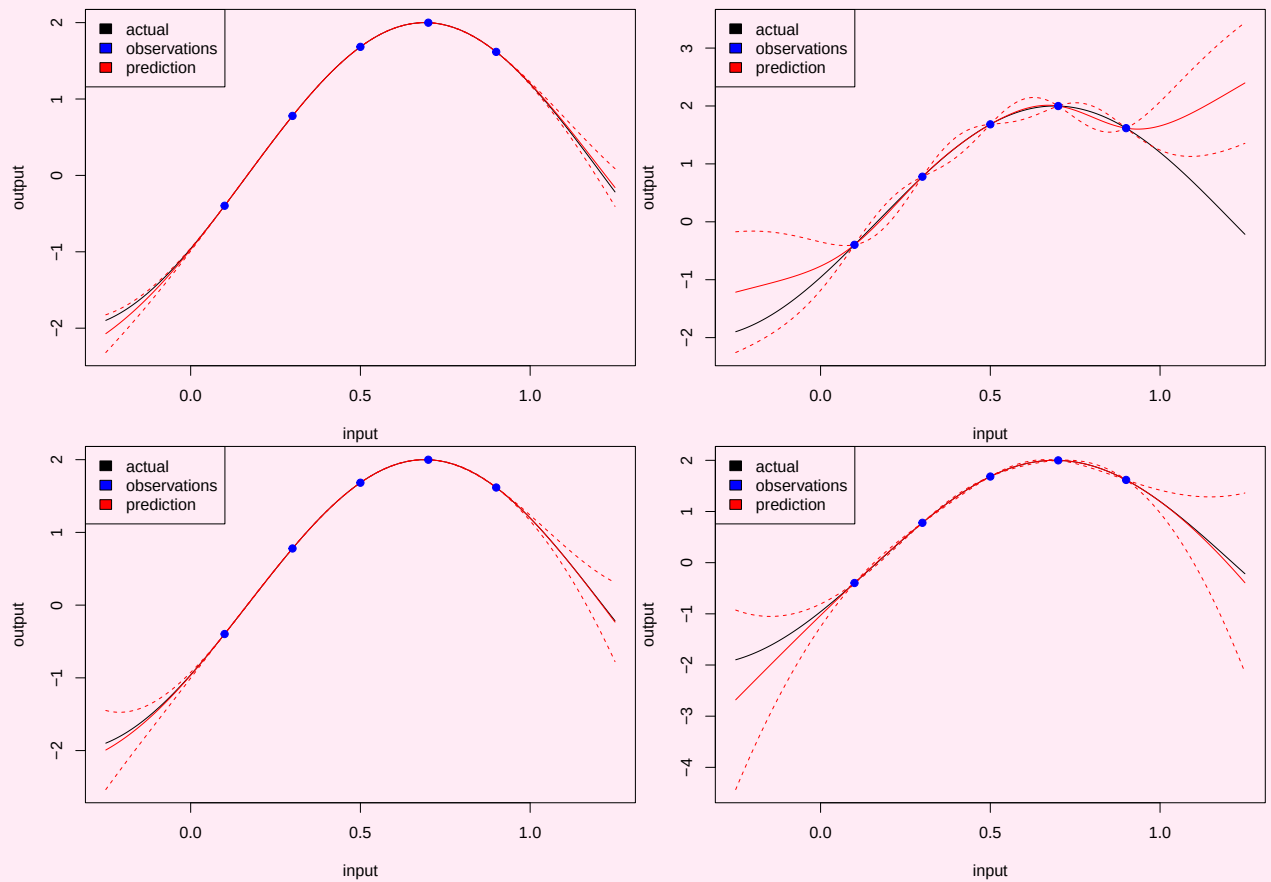


Figure 2: Top: DiceKriging with Gauss kernel (left) and Matern 5/2 (right). Bottom: RobustGaSP with Gauss kernel (left) and Matern 5/2 (right).

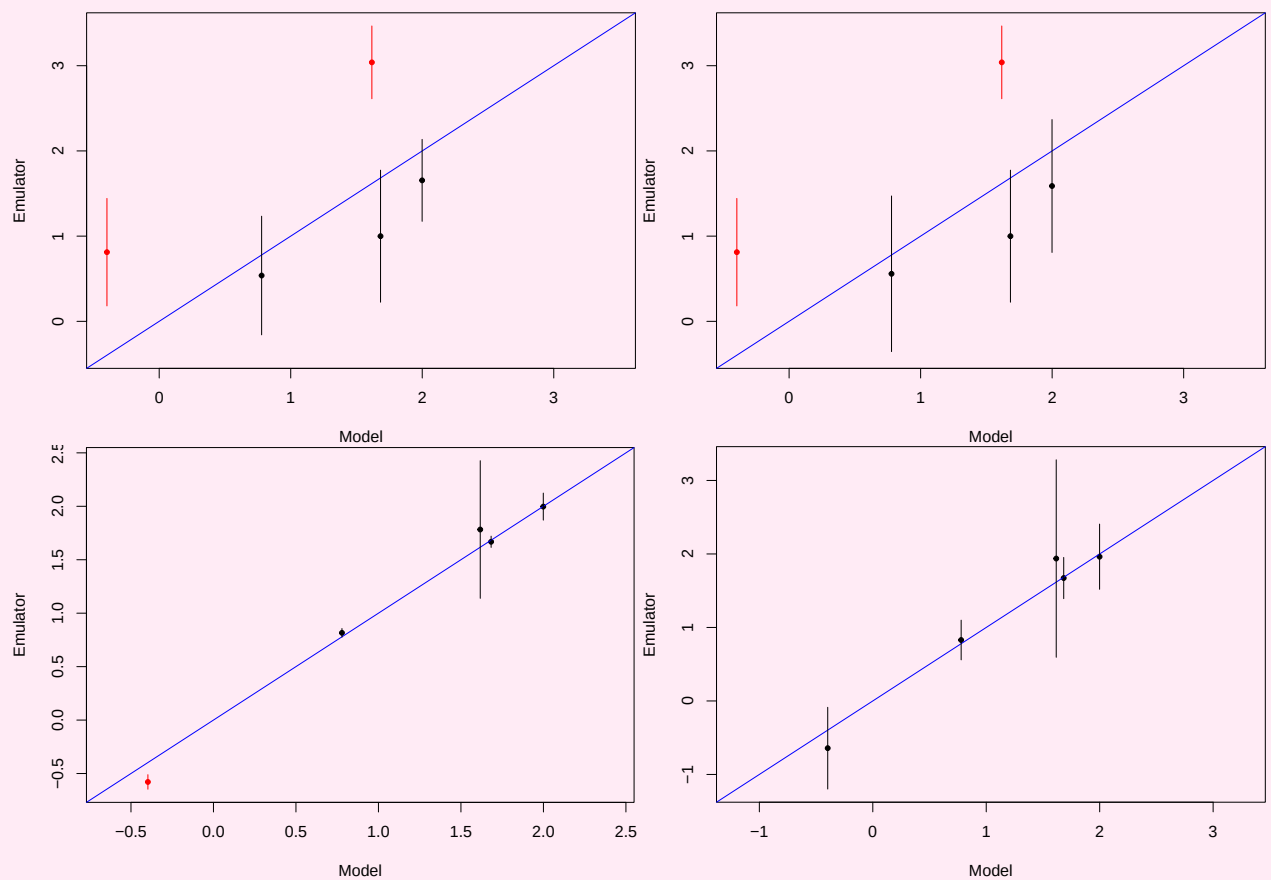


Figure 3: LOO cross validation. Top: DiceKriging with Gauss kernel (left) and Matern 5/2 (right). Bottom: RobustGaSP with Gauss kernel (left) and Matern 5/2 (right).

Going forward, for now, only the Gauss kernel is dealt with; Matern 5/2 will be returned to once I’ve figured out the estimates returned by the two packages and the derivative of the Matern 5/2 kernel.

The estimates returned by both packages using the Gauss kernel are shown in Table 2. For **RobustGaSP**, the Manual indicates that it returns “inverse-range parameters”, so the inverse of these values have been found, resulting in ℓ_{RK} . In the Gauss kernel case, the Manual for **RobustGaSP** indicates that the denominator of the fraction is ℓ_{RK}^2 whereas for **DiceKriging** it’s $2\ell_{DK}^2$. To facilitate direct comparison of the parameter estimates, we require a range parameter for **RobustGaSP**, ℓ , which when plugged into $2\ell^2$ results in ℓ_{RG}^2 . Therefore

$$\ell = \frac{\ell_{RG}}{\sqrt{2}}$$

since

$$2 \left(\frac{\ell_{RG}}{\sqrt{2}} \right)^2 = 2 \frac{\ell_{RG}^2}{2} = \ell_{RG}^2.$$

Table 6: Estimates table

Package	Kernel	Trend 1	Trend 2	Range	Variance
DiceKriging	Gauss	-1.13	0.18	0.69	5.05
RobustGaSP	Gauss	-0.7	-0.57	0.79	15.1

Verifying the estimates by-hand for both packages

To calculate the posterior predictions at the 1000 testing points by-hand, we require the matrix H containing the inputs for the 5 observations and well as matrix A , its inverse, and $\mathbf{t}(x^*)$ for all 1000 of the new inputs x^* . A and $\mathbf{t}(x^*)$ are built using the estimate of the range parameter returned by each package. By-hand predictions match those returned by **DiceKriging** (Figure 2) and **RobustGaSP** (Figure 3).

For both we calculate

$$\frac{\sum_{i=1}^{1000} \left| (\text{predictions from package})_i - (\text{predictions by-hand})_i \right|}{1000}$$

then divide this by 10^{-8} (machine precision) to get 0.0006 for **DiceKriging** and 0.002 for **RobustGaSP** (the latter approximately a factor of 3 larger than the former).

Why don’t **RobustGaSP**’s estimates change? What is the stochastic element of **DiceKriging**?

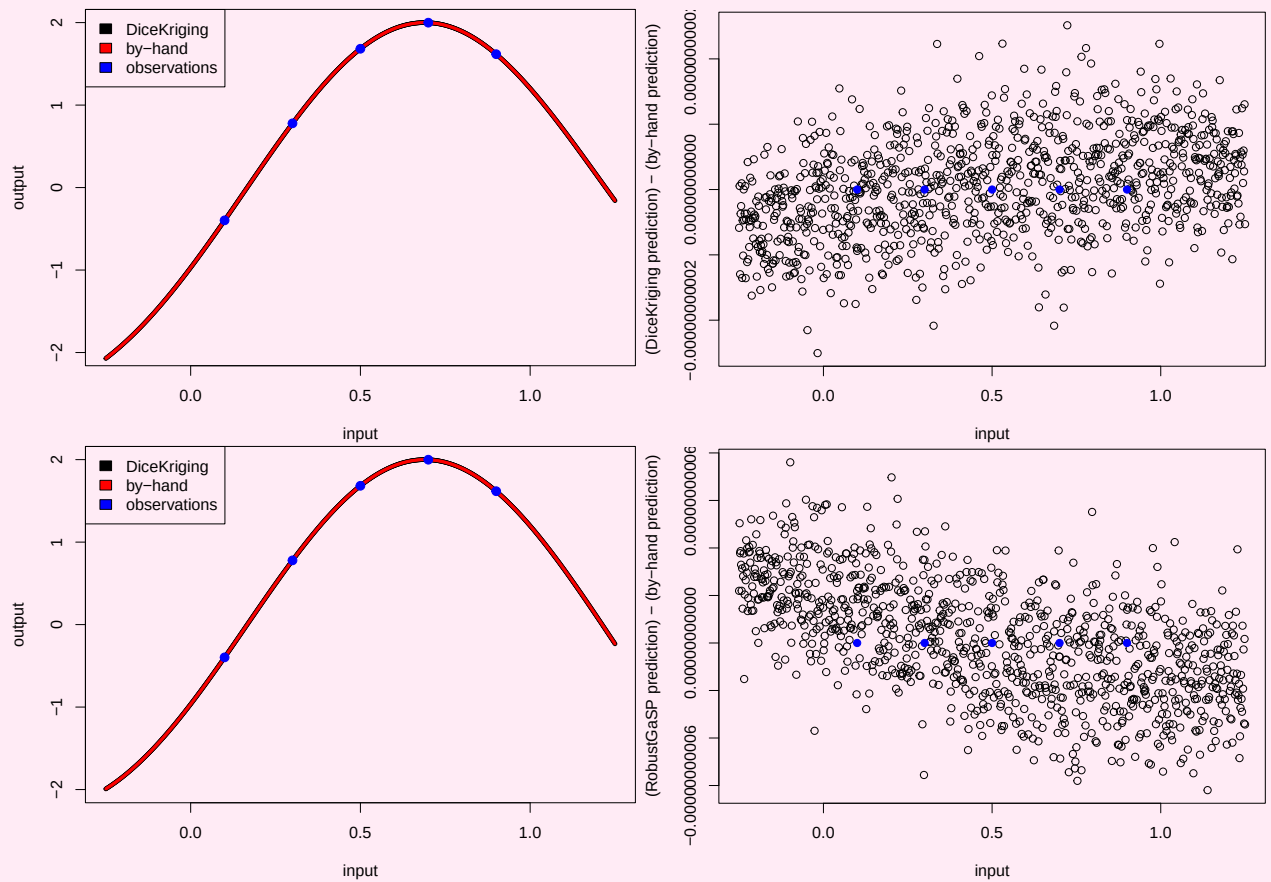


Figure 4: The predictions found by-hand match well with those made by both packages (left), with the latter minus the former found to be essentially zero (right). Top: DiceKriging. Bottom: RobustGaSP.